Linux File System

A Linux file system is a structured collection of files on a disk drive or a partition. A partition is a segment of memory and contains some specific data. In our machine, there can be various partitions of the memory. Generally, every partition contains a file system.

The general-purpose computer system needs to store data systematically so that we can easily access the files in less time. It stores the data on hard disks (HDD) or some equivalent storage type. There may be below reasons for maintaining the file system:

- Primarily the computer saves data to the RAM storage; it may lose the data if it gets turned off. However, there is non-volatile RAM (Flash RAM and SSD) that is available to maintain the data after the power interruption.
- Data storage is preferred on hard drives as compared to standard RAM as RAM costs more than disk space. The hard disks costs are dropping gradually comparatively the RAM.

The Linux file system contains the following sections:

- The root directory (/)
- A specific data storage format (EXT3, EXT4, BTRFS, XFS and so on)
- A partition or logical volume having a particular file system.

What is the Linux File System?

Linux file system is generally a built-in layer of a Linux operating system used to handle the data management of the storage. It helps to arrange the file on the disk storage. It manages the file name, file size, creation date, and much more information about a file.

If we have an unsupported file format in our file system, we can download software to deal with it.

Linux File System Structure

Linux file system has a hierarchal file structure as it contains a root directory and its subdirectories. All other directories can be accessed from the root directory. A partition usually has only one file system, but it may have more than one file system.

The file system requires an API (Application programming interface) to access the function calls to interact with file system components like files and directories. API facilitates tasks such as creating, deleting, and copying the files. It facilitates an algorithm that defines the arrangement of files on a file system.

The first two parts of the given file system together called a **Linux virtual file system**. It provides a single set of commands for the kernel and developers to access the file system. This virtual file system requires the specific system driver to give an interface to the file system.

Directory Structure

The directories help us to store the files and locate them when we need them. Also, directories are called folders as they can be assumed of as folders where files reside in the form of a

physical desktop analogy. Directories can be organized in a tree-like hierarchy in Linux and several other operating systems.
The below table gives a very short standard, defined, and well-known top-level Linux directory list and their purposes:

- **/ (root filesystem):** It is the top-level filesystem directory. It must include every file needed to boot the Linux system before another filesystem is mounted. Every other filesystem is mounted on a well-defined and standard mount point because of the root filesystem directories after the system is started.
- **/boot:** It includes the static kernel and bootloader configuration and executable files needed to start a Linux computer.
- **/bin:** This directory includes user executable files.
- **/dev:** It includes the device file for all hardware devices connected to the system. These aren't device drivers; instead, they are files that indicate all devices on the system and provide access to these devices.
- **/etc:** It includes the local system configuration files for the host system.
- **/lib:** It includes shared library files that are needed to start the system.
- **/home:** The home directory storage is available for user files. All users have a subdirectory inside /home.
- **/mnt:** It is a temporary mount point for basic filesystems that can be used at the time when the administrator is working or repairing a filesystem.
- **/media:** A place for mounting external removable media devices like USB thumb drives that might be linked to the host.
- **/opt:** It contains optional files like vendor supplied application programs that must be placed here.
- **/root:** It's the home directory for a root user. Keep in mind that it's not the '/' (root) file system.
- **/tmp:** It is a temporary directory used by the OS and several programs for storing temporary files. Also, users may temporarily store files here. Remember that files may be removed without prior notice at any time in this directory.
- **/sbin:** These are system binary files. They are executables utilized for system administration.
- **/usr:** They are read-only and shareable files, including executable libraries and binaries, man files, and several documentation types.
- **/var:** Here, variable data files are saved. It can contain things such as MySQL, log files, other database files, email inboxes, web server data files, and much more.

Linux File System Features

In Linux, the file system creates a tree structure. All the files are arranged as a tree and its branches. The topmost directory called the **root (/) directory**. All other directories in Linux can be accessed from the root directory.
Some key features of Linux file system are as following:

- **Specifying paths:** Linux does not use the backslash (\) to separate the components; it uses forward slash (/) as an alternative. For example, as in Windows, the data may be stored in C:\ My Documents\ Work, whereas, in Linux, it would be stored in /home/ My Document/ Work.
- **Partition, Directories, and Drives:** Linux does not use drive letters to organize the drive as Windows does. In Linux, we cannot tell whether we are addressing a partition, a

network device, or an "ordinary" directory and a Drive.

- **Case Sensitivity:** Linux file system is case sensitive. It distinguishes between lowercase and uppercase file names. Such as, there is a difference between test.txt and Test.txt in Linux. This rule is also applied for directories and Linux commands.
- **File Extensions:** In Linux, a file may have the extension '.txt,' but it is not necessary that a file should have a file extension. While working with Shell, it creates some problems for the beginners to differentiate between files and directories. If we use the graphical file manager, it symbolizes the files and folders.
- **Hidden files:** Linux distinguishes between standard files and hidden files, mostly the configuration files are hidden in Linux OS. Usually, we don't need to access or read the hidden files. The hidden files in Linux are represented by a dot (.) before the file name (e.g., .ignore). To access the files, we need to change the view in the file manager or need to use a specific command in the shell.

Types of Linux File System

When we install the Linux operating system, Linux offers many file systems such as **Ext, Ext2, Ext3, Ext4, JFS, ReiserFS, XFS, btrfs,** and **swap**.

Let's understand each of these file systems in detail:

1. Ext, Ext2, Ext3 and Ext4 file system

The file system Ext stands for **Extended File System**. It was primarily developed for **MINIX OS**. The Ext file system is an older version, and is no longer used due to some limitations.

Ext2 is the first Linux file system that allows managing two terabytes of data. Ext3 is developed through Ext2; it is an upgraded version of Ext2 and contains backward compatibility. The major drawback of Ext3 is that it does not support servers because this file system does not support file recovery and disk snapshot.

Ext4 file system is the faster file system among all the Ext file systems. It is a very compatible option for the SSD (solid-state drive) disks, and it is the default file system in Linux distribution.

2. JFS File System

JFS stands for **Journaled File System**, and it is developed by **IBM for AIX Unix**. It is an alternative to the Ext file system. It can also be used in place of Ext4, where stability is needed with few resources. It is a handy file system when CPU power is limited.

3. ReiserFS File System

ReiserFS is an alternative to the Ext3 file system. It has improved performance and advanced features. In the earlier time, the ReiserFS was used as the default file system in SUSE Linux, but later it has changed some policies, so SUSE returned to Ext3. This file system dynamically supports the file extension, but it has some drawbacks in performance.

4. XFS File System

XFS file system was considered as high-speed JFS, which is developed for parallel I/O processing. NASA still using this file system with its high storage server (300+ Terabyte server).

5. Btrfs File System

Btrfs stands for the **B tree file system**. It is used for fault tolerance, repair system, fun administration, extensive storage configuration, and more. It is not a good suit for the production system.

6. Swap File System

The swap file system is used for memory paging in Linux operating system during the system

hibernation. A system that never goes in hibernate state is required to have swap space equal to its RAM size.

Linux File System - inode (index node)

inode (or index node) is a data structure that describes the linux file-system of one object such as a file or a directory.

They are also known as filesystem objects (e.g. files and directories).

Each node stores:

- the attributes
- and disk block locations of the object.

The i-number is a integer that represents the index (ie the location) of the node element in the table storing all nodes (the table is stored as an array)

Node.js is a JavaScript runtime built on Chrome's V8 JavaScript engine. Node.js can be installed in multiple ways on your Ubuntu Linux machine. You can use **Ubuntu's official repository** to install Node.js or another way to use **NodeSource repository**. Installation via NodeSource repository will allow you to choose latest version of Node.js.

Installing Node On Ubuntu 18.04 and 16.04: There are two methods Ubuntu official repository and NodeSouce repository to install Node.js on Ubuntu.

Install Node.js using Ubuntu official repository: Node.js is available in Ubuntu's repository and you can easily install it using a few commands. Follow the steps below to install Node.js on your Ubuntu operating system.

- **Step 1:** Open your terminal or press Ctrl + Alt + T.
- **Step 2:** To install node.js use the following command:

  sudo apt install nodejs

1. Hardware Requirements

Below are the hardware requirements for installing Node.js on Ubuntu.

- Node.js does not need a complex hardware configuration to work; most machines nowadays are compatible.
- Node.js installation requires a stable hard drive for it to work.

- You should have the necessary prerequisites to install Node.js on Ubuntu.
- Virtually all modem computers can run Node.Js, including miniature computers like BeagleBone or Arduino YÚN.
- The Random-Access Memory (RAM) of your system should be up to 4GB and your system storage should be at least 256GB of Hard Disk Space.

2. Software Requirements

- Below are the software requirements for installing Node.js on Ubuntu. The web browser should be intact. Any browser such as Google Chrome, Mozilla Firefox, or Microsoft Edge can run Node.js.
- Operating System should be accurate and free, having an Ubuntu server installed with a non-root Sudo user and firewall.

How to Install Node.js on Ubuntu [Step-by-Step]

Follow the three methods below to install Node.js on your Ubuntu machine.

Method 1: Installing the Stable Version for Ubuntu

Before you can install a stable version of Node.js for Ubuntu, you must update the terminal first.
Step 1: You can use the apt package manager to obtain the version that can run Node.js. Update the index for your regional package usage.

```
$ sudo apt update
```

Copy Code

Step 2: Install Node.js from the repository in step two.

```
$ sudo apt install Node.js
```

Copy Code

Step 3: Install npm ubuntu with the code below.

```
$ sudo apt install npm
```

Copy Code

Step 4: Use this to check which Node.js version you installed.

```
$ Node.js -v && npm --version
```

Copy Code

And there you have it for this method. If you want to upgrade your JavaScript skills, visit the Full Stack Developer course to enhance your development skill.

Method 2: Install Node.js Using a PPA

Software repositories of various packages specifically created for Ubuntu 20.14 users are included in the PPA repository. The PPA repository for the most recent Node.js version is added first.
Before installing Node.js using PPA, ensure that you have the curl command-line utility installed on your system. If curl isn't installed on your system paste this command on your terminal to install curl:

```
$ sudo apt install curl
```

File descriptor

It is simply an index into the file descriptor table. For each process in our operating system, there is a **process control block**(PCB). PCB keeps track of the context of the process. So one of the fields within this is an array called file descriptor table.
This array keeps track of all the resources that the process owns and can operate on. The file descriptor table holds pointers to the resources. Resources could be

- File
- Terminal I/O
- Pipes
- Socket for the communication channel between machines
- Any devices

So if any process opens or uses any resources will have an entry in the file descriptor table. Any process when it first starts is given access to three resources. These resources are:

- stdin
- stdout
- stderr

Understand this with an example. Suppose we are working on the terminal then input for it is stdin, the output is stdout and the error is stderr. And if we open any other terminal then both are two different processes now. The newly opened terminal will have its own stdin, stdout because it's a different process.

The file descriptor is just an integer that you get from the **open()** system call.
Example of file descriptor:
int fd = open(filePath, mode);
File pointer is a pointer returned by **fopen()** library function. It is used to identify a file. It is passed to a **fread()** and **fwrite()** function.
Example of file pointer:

```
FILE *fp;
fp = fopen("sample.txt,"a");
fprintf( fp, "Welcome to GFG");
fclose(fp);
```

File pointer provides high efficiency and high portability. It is buffered, unlike the file descriptor. Generally, we use a file pointer when we are reading from a file.

Following is a table of differences between File Descriptor and File Pointer

	File Pointer	File Descriptor
1.	File pointer is allocated with fopen function call FILE *fp; fp = fopen("sample.txt,"a");	File descriptor is allocated with open system call int fd = open(filePath, mode);
2.	It is generally, use for the application which is doing extensive read or write from a file	It is generally used for the application that do frequently random access of file
3.	It is a pointer	It is an integer value like 0, 1, 2
4.	The file pointer is buffered	The file descriptor is not buffered
5.	It is highly portable and efficient	It is less portable and efficient in comparison with the file pointer

6.	It is passed to fread() and fwrite() function	It is passed to read() and write() function
7.	It is not suitable for inter-process communication	It is suitable for inter-process communication
8.	Library functions generally use file pointer	System call generally use the file descriptor
9	file pointers are data structures.	File descriptors are represented as integer values
10	file pointers are used to represent the current position within the file.	File descriptors are used to identify an open file,
11	file pointers are used by the application program to read from or write to the file.	File descriptors are used by the operating system to perform operations on the file,
12	file pointers can have any value depending on the position within the file.	File descriptors are non-negative integers,

Device Drivers in Linux

Drivers are used to help the hardware devices interact with the operating system. In windows, all the devices and drivers are grouped together in a single console called device manager. In Linux, even the hardware devices are treated like ordinary files, which makes it easier for the software to interact with the device drivers. When a device is connected to the system, a device file is created in **/dev** directory.

Most Common types of devices in Linux:

1. **Character devices** – These devices transmit the data character by characters, like a mouse or a keyboard.
2. **Block devices** – These devices transfer unit of data storage called a *block,* USB drives, hard drives, and CD ROMs

To list all the device files use the below command.

```
ls -l /dev
```

Disk and Driver Commands

1. fdisk – It stands for format disk. This command is used to display the partitions on a disk and other details related to the file system.

```
sudo fdisk -l
```

2. sfdisk – This command displays the partitions on the disk, the size of each partition in MB, etc.

3. parted – This command helps list and modify the partitions of the disk.

```
sudo parted -l
```

4. df – Displays the details of the file system. Using grep we can filter real hard disk files.

```
df -h | grep ^/dev
```

5. lsblk – List details about the block devices.

```
lsblk
```

6. inxi – Lists details about the hardware components in the file system.

```
inxi -D -xx
```

The **kernel** is a monolithic piece of software, but it is also responsible to provide support to the hardware. Most of the devices have built-in kernel modules, so when they are plugged in, they start working automatically.

The System Call

The system call is a mechanism in Linux that allows user-space applications to connect with the kernel, which is also a component of what makes up the operating system's core. A user-space application needs to send a system call to the kernel that is used to have a privileged functioning performed on its behalf, including writing or reading information to a file or beginning a new process. In this article, we will be discussing the Linux System Call in detail along with its various types.

How Linux System Calls Work?

The system calls are carried out in the kernel as performs that consumer-space applications can access through the C library's standard library functions. Among them are open(), read(), write(), close(), fork(), exec(), and numerous other functions.

Types of System Calls in Linux

System calls in Linux are classified into five types based on the operation they perform. These are the categories −

- **Process management system calls** − These system calls are used to manage processes, such as starting new ones, stopping existing ones, and waiting for them to finish. Fork(), exec(), wait(), and exit() are all examples of process management system calls ().
  fork() − This system call duplicates the calling process to create a new process. The child process is the new process that runs the same program as the parent process.
  exec() − The system call exec() is used to replace the current process image with a new process image. It loads the specified program into memory and executes it.
  wait() − A parent process uses this system call to wait for its child process to terminate. The parent process is halted until the child process completes.
  exit() − The present-day process is terminated and a status code is returned to the originating process.
- **File management system calls** − These calls to the system are used to open, read, write, and close documents, as well as to create, rename, and delete them. Some file management system calls () are open(), read(), write(), close(), mkdir(), and rmdir().
  open() − broadens a file and goes back to a file descriptor (an integer identifying the open file.
  read() − The above system call examines data from an open file into a memory buffer.
  write() − It is an internal call that is used to write information stored in a buffer of memory to an open file.
  close() − It is a system call for closing a file determined by its file descriptive term.
- **Device management system calls** − These system calls are used to manage I/O devices, such as reading from and writing to them, setting device attributes, and controlling device operations. System calls for device management include read(), write(), ioctl(), and select ().
  read() − This system call reads data from an input device into a memory buffer.
  write() − It is a system call that is used to write data from a memory buffer to an output device.
  ioctl() − This system call controls the behavior of a device by setting or retrieving device attributes.

select() − This system call is used to wait for I/O operations to complete on multiple devices.

- **Network management system calls** − These system calls are used to manage network resources such as connecting and disconnecting from networks, sending and receiving data over networks, and resolving network addresses. Socket(), connect(), send(), and recv() are examples of network management system calls ().
  socket() − It establishes a socket, and this is a network of things endpoint which can be used to communicate.
  connect() − It is an internal call used to establish a connection to an external network endpoint.
  send() − It is an application call used to transfer data across a computer's connection.
  recv() − A network connection is used to send data to this system call.
- **System information system calls** − Such system calls are utilized to query and change system parameters such as time, configuration, and performance statistics. System information system calls () include getpid(), getuid(), gethostname(), and sysinfo().
  getpid() − The program ID of the process that is currently running is returned by this system call.
  getuid() − The system call in question is used to get the user ID of the current process.
  gethostname() − This call to the system is used to get the hostname of the system.
  sysinfo() − The above system call returns information about the system, such as the amount of memory free and the overall amount of processes.

Different Ways to Create File in Linux

There are mainly six ways of creating files in Linux. All of them have their own purpose and benefits. They are as follows:

Creating file using the `cat` command in Linux

It is the most universal command/tool for creating files on Linux systems. We cannot edit a file using the cat command. Major operations that can be done using it are as follows:

To create files and write the data into them.

```
cat >file1
```

This command creates a new file file1 (in write mode) if it doesn't exist in the present working directory. If any file with file name file1 exists in the current directory, it is overwritten.
Note: After writing the text into the file, press ctrl+d to save and exit from the writing mode.

touch` command in Linux

We can create an empty file (or multiple empty files) using this command. But its main purpose is to change or update the time-stamp of a file. Major operations that can be done using it are as follows:

Creating a file using touch in Linux

```
touch file2
```

touch file2
Used `ls` command to list files in the current directory and used `cat` command to see the content inside a file.

Creating multiple files at same time using `touch` command in Linux

```
touch file_1 file_2 file_3 file_4
```

touch file_1 file_2 file_3 file_4

How to display timestamp of files in Linux

```
ls -l
```

ls -l

How to change timestamp of a file in Linux

For example: If we want to change the timestamp of a file_name `file_1`. We use command as follows.

```
touch file_1
```

touch file_1

Using `ls -l` command to display timestamp of files.

Creating File using `vi` or `vim` command in Linux

Its main function is to edit files. It is commonly used by programmers to edit the textual content of any file on vi text editor. Major operations that can be done using it are as follows:

Note: To save and exit from the vi-text editor, press the **Escape key** and then type **:wq** and hit enter.

Creating a file using vi in Linux

```
vi file_1
```

This command creates a new file file_1 and opens it on the vi-text editor if it doesn't exist in the present working directory. If a file with the file name file_1 exists in the current directory, then this command just opens the file on the vi-text editor.

vi file_1

Used `ls` command to list files in the current directory and used `cat` command to see the content inside a file.

Creating a file using vim in Linux

This command creates a new file file_1 and opens it on the vim-text editor if it doesn't exist in the present working directory. If a file with the file name file_1 exists in the current directory, then this command just opens the file on the vm-text editor. Vim is the update version of vi text editor.

```
vim file_2
```

vim file_2

Used `ls` command to list files in the current directory and used `cat` command to see the content inside a file.

Creating file using `nano` command in Linux

It may/may not be found in all distributions of LINUX. We can create as well as edit files.

Note: To exit nano Text Editor press **ctrl + x.**

```
nano file_1
```

nano file_1

Used `ls` command to list files in the current directory and used `cat` command to see the content inside a file.

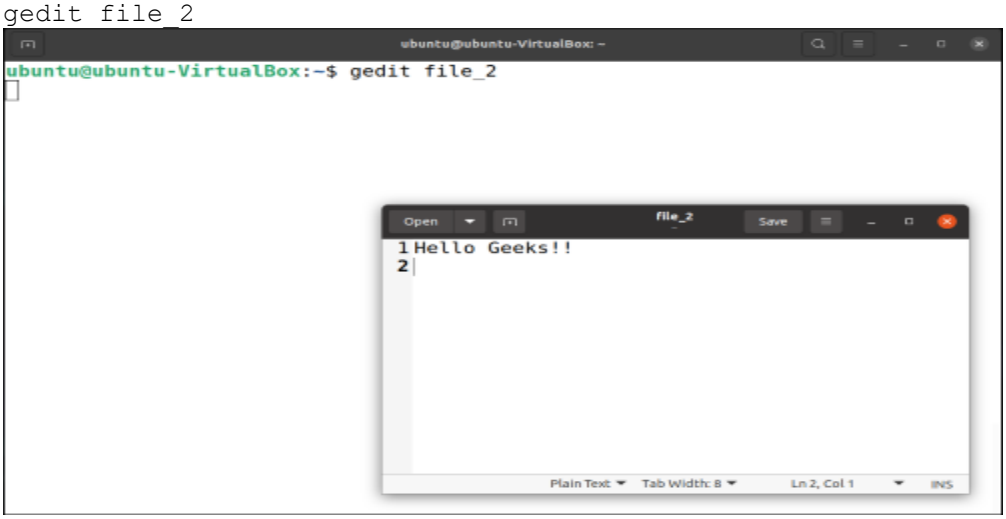
Creating a file using `gedit` command in Linux

Linux's users normally use the command line interface (CLI) for writing or editing the text files. But if we want to edit a text file graphically on Linux machines without learning about the powerful editors like vim and nano then gedit text editor makes it easier for us.

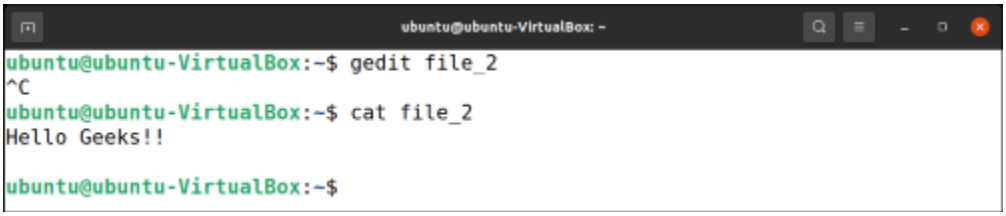
"gedit" stands for GNOME text editor, it's a standard default text editor found in any system with a GNOME desktop environment including Ubuntu, Fedora, Debian, CentOS, and Red Hat. Using gedit we can create as well as write/edit the text files.

Create a file.

```
gedit file_2
```



1.



2.

This command creates a new file file_2 (in write mode) on the gedit text editor if it doesn't exist in the present working directory. If any file with file name file_2 exists in the current directory, then it is opened (in edit mode) on the gedit text editor.

Note: To use the terminal again, press **ctrl + c**.

Open and edit the file.

As we create or edit a file using gedit command, by default the file gets open on the gedit text editor and wait for it to close before it returns you to the terminal prompt. If you want to use the terminal window while the gedit text editor is open, launch gedit with the following command instead.

```
gedit file_2 &
```

This command opens gedit text editor as a background task. We get the command line prompt back straight away and carry on using the terminal window even when gedit text editor is running.

Open and edit the file

mv command

We normally use mv command to move the files or directories from one place to another in Linux systems. But we can also use it to create new files with the contents of some other file on the system.

```
mv file_2 file_3
```

This command creates a new file file_3 with the contents of file_2 if file_3 doesn't exist in the present working directory. If any file with file name file_3 exists in the current directory, then it is overwritten with the contents of file_2.

mv command

Note: This command copies the content of file_2 to file_3 and **deletes** file_2.

Creating file using `printf` command in Linux

The printf command can be used to create a file with formatted text. To create a file using the printf command

Open the terminal and type the following command:

```
printf "txt_we_want"> File_name
```

Example:

If we want to write "hello connections" and want to create a file_name "file_1"

```
printf "hello connections"> file_1
```

ls` command to list files in the current directory and used `cat` command to see the content inside a file.

Open File in Linux

There are various ways to open a file in a Linux system. It is a fairly straight forward process to view the contents of a file, but if you are a new user, it may bother you. It is not as easy as opening a file in Notepad. From the Linux terminal, you must have some exposures to the Linux basic commands. There are some commands such as **cat, ls**, that are used to read files from the terminal.

In Linux, we can display various file formats such as **text file, audio files, video, image, doc, pdf, or any other file contents.**

Open File Using cat Command

This is the most popular and easy way to display the file content. It simply prints the file content to the terminal. It provides many options to make it more specific. To go in-depth with cat command, visit Linux cat.

To display the file content, execute the cat command as follows:

1. cat <file name>

Let's create a file to understand how to open a file. Execute the below command:

1. cat > Test.txt
2. This is a Test file.

`lseek` function

The `lseek` function in Linux is used to change the file offset (position) within an open file. It allows you to move the read/write pointer to a specific position within the file. This is particularly useful for random access and updating specific parts of a file. Here's how the `lseek` function is used:

Syntax:

```
off_t lseek(int fd, off_t offset, int whence);
```

Parameters:

- `fd`: The file descriptor of the open file.
- `offset`: The number of bytes to move the file offset. The value can be positive or negative.
- `whence`: The reference point for the offset change. It can take one of three values:

— `SEEK_SET`: Set the offset to the provided value.

— `SEEK_CUR`: Move the offset from the current position.

— `SEEK_END`: Set the offset from the end of the file.

Return Value:

- On success, the `lseek` function returns the new offset from the beginning of the file.
- On failure, it returns `-1`, and you can use the `errno` variable to determine the specific error.

Linux File Links

A Linux filesystem has many hard links and symbolic links. A link is a connectivity between the filename and the actual data byte in the disk space. More than one filename can **link** to the same data.

There are two types of links in Linux OS:

1. Hard Links
2. Soft Links

---

1) Hard Links

They are the low-level links. It links more than one filename with the same Inode and it represents the physical location of a file.

When hard link is created for a file, it directly points to the Inode of the original file in the disk space, which means no new Inode is created. Directories are not created using hard links and they can not cross filesystem boundaries. When the source file is removed or moved, then hard links are not affected.

2) Soft Links (Symbolic Links)

Soft links are very common. It represents a virtual or abstract location of the file. It is just like the shortcuts created in Windows. A soft link doesn't contain any information or content of the linked file, instead it has a pointer to the location of the linked file. In other words, a new file is created with new Inode, having a pointer to the Inode location of the original file.

It is used to create link between directories and can cross filesystem boundaries. When the source file is removed or moved, then soft links are not updated.

In Linux/UNIX, a **symbolic link or soft link**, also referred to as a **symlink**, is a useful tool for linking files or directories from different locations. Similar to a pointer in C programming, a symlink directs to the original file from an alternate location. The creation of symbolic links is facilitated by the ln command. However, it is important to note that a symlink will only function if the original file exists on the system. If the original file is accidentally deleted, the symlink file becomes unusable.

Advantages of Symlink

Symlink has many more advantages making it more reliable and efficient when it comes to usage.

- It is more efficient in linking files across the file system.
- Symlink has a special case that it can also create a link file of a directory.
- It creates multiple access points for a file without having original access copies of a file.
- Kernel checks the filename of a link file and directly goes to the original file like if kernel traverses the namespace for the user.

Disadvantages of Symlink

Symlink has some disadvantages when it comes to usage.

- The most disadvantage of a symlink is that it does not directly link to a file.
- There is no use of the symlink file if the original file gets deleted or changed.
- The possibility of symlink can be dead or dangling after creating multiple files.

Soft links vs Hard links

Basis	Soft links	Hard links
Inode number	Soft links have different inode numbers.	Hard links have the same inode number.
File creation	Soft links can be created for files and directories.	Hard links cannot be created for the directory.
Data	Soft links can be only used until the original files and directories are present.	Hard links can be used after the deletion of the file.
File system	Soft links can be used across the file system.	Hard links cannot be used across the file system.
File permission	Original file permission (-rw-r–r–) and Link file permission (lrwxrwxrwx) are different in soft links.	Both files have the same permissions in Hard links.

How to Symlink a file in Linux?

Step 1: To symlink a file, first, we need to create a file named “gfgfile”

```
touch gfgfile
```

`touch` command is used to create a file.

touch

Step 2: For creating a symlink file, we can use the command as:

```
ln -s [original file] [symbolic link file]
```

- **ln**: make a link between files.
- **-s**: create a symbolic link of a file instead of a hard link.

Unlink command

The unlink command is used for removing files and links. You can use it to delete both soft and hard links.
It is one of the simplest commands that have no options other than `--help` and `--version`.
To delete a file with unlink, use it like this:

```
unlink filename
```

To delete a link:

```
unlink link_name
```

stat command

The **stat** is a command which gives information about the file and filesystem. Stat command gives information such as the size of the file, access permissions and the user ID and group ID, birth time access time of the file. Stat command has another feature, by which it can also provide the file system information. This is the best tool to use when we want the information of any file. Now let' see one by one how can we use the stat command.

Using stat command

The basic syntax of using stat command is as follows:

```
stat --options filenames
```

Stat can accept one or more filenames as an input to the stat command. Now let's see one example

```
stat /etc/resolv.conf
```

Diffrence between stat,fstat and lstat

These functions return information about a file. No permissions are required on the file itself, but — in the case of **stat**() and **lstat**() — execute (search) permission is required on all of the directories in *path* that lead to the file.
stat() stats the file pointed to by *path* and fills in *buf*.
lstat() is identical to **stat**(), except that if *path* is a symbolic link, then the link itself is stat-ed, not the file that it refers to.
fstat() is identical to **stat**(), except that the file to be stat-ed is specified by the file descriptor *filedes*.
All of these system calls return a *stat* structure, which contains the following fields:
In Unix operating systems, the **chmod** command is used to change the access mode of a file. The name is an abbreviation of **change mode**. Which states that every file and directory has a set of permissions that control the permissions like who can read, write or execute the file. In this the permissions have three categories: read, write, and execute simultaneously represented by `r`, `w` and `x`. These letters combine together to form a specific permission for a group of users.

chmod command

The `chmod` command is used to modify this permission so that it can grant or restrict access to directories and files. Let's have a look at the syntax and options for the `chmod` command in Linux Operating System.

Syntax:

```
chmod [options] [mode] [File_name]
```

"chmod" in Linux [options]

Options	Description
`-R`	Apply the permission change recursively to all the files and directories within the specified directory.
`-v`	It will display a message for each file that is processed. while indicating the permission change that was made.
`-c`	It works same as **`-v`** but in this case it only displays messages for files whose permission is changed.
`-f`	It helps in avoiding display of error messages.
`-h`	Change the permissions of symbolic links instead of the files they point to.

Note: Options in **`chmod`** are basically used for making changes in bulk and modifying permissions across multiple files or directories at once.

"chmod" in Linux [mode]

The "mode" helps in setting new permissions that have to be applied to files or directories. This mode can be specified in several ways, we will discuss two modes: Symbolic and Octal mode.

1) Symbolic mode

If we talk about symbolic mode, we can say that it is the most common method used for specifying fir permissions. In this we have to make a combination of **letters** and **operators** to set or tell what to do with permissions.

The following operators can be used with the symbolic mode:

Operators	Definition
`+`	Add permissions
`-`	Remove permissions
`=`	Set the permissions to the specified values

The following letters that can be used in symbolic mode:

Letters	Definition
`r`	Read permission
`w`	Write permission
`x`	Execute permission

The following Reference that are used:

Reference	Class

Reference	Class
u	Owner
g	Group
o	Others
a	All (owner,groups,others)

Examples of Using the Symbolic mode:

- Read, write and execute permissions to the file owner:

```
chmod u+rwx [file_name]
```

- Remove write permission for the group and others:

```
chmod go-w [file_name]
```

- Read and write for Owner, and Read-only for the group and other:

```
chmod u+rw,go+r [file_name]
```

2) Octal mode

It is also a method for specifying permissions. In this method we specify permission using three-digit number. Where..

- **First digit** specify the permission for Owner.
- **Second digit** specify the permission for Group.
- **Third digit** specify the permission for Others. The digits

NOTE: The digits are calculated by adding the values of the individual permissions.

Value	Permission
4	Read Permission
2	Write Permission
1	Execute Permission

Examples of Using the Octal mode:

Suppose if we to give read and write permission to the file Owner. Read, write and executable permission to the Group. Read-only permission to the Other. They our command would be.

```
chmod 674 [file_name]
```

Here.

- **6** represent permission of file Owner which are **(rw).**
- **7** represent permission of Group which are **(rwx).**
- **4** represent permission of Other which is **(r).**

chown command

Different users in the operating system have ownership and permission to ensure that the files are secure and put restrictions on who can modify the contents of the files. In Linux, there are different users who use the system:

- **Root User:** It is a superuser who has access to all the directories and files in our system and it can perform any operation. An important thing to note is that only the root user can perform changing of permissions or ownerships of the files that are not owned by them.
- **Regular User:** These users have limited access to files and directories and can only modify the file that they own.

Each *user* has some properties associated with them, such as a user ID and a home directory. We can add users to a group to make the process of managing users easier. A *group* can have zero or more users. A specified user can be associated with a "default group". It can also be a member of other groups on the system as well.

Ownership and Permissions: To protect and secure files and directories in Linux we use permissions to control what a user can do with a file or directory. Linux uses three types of permissions:

- **Read:** This permission allows the user to read files in directories, it lets the user read directories and subdirectories stored in it.
- **Write:** This permission allows a user to modify and delete a file. Also, it allows a user to modify its contents (create, delete, and rename files in it) for the directories. Unless the execution permission is given to directories changes do affect them.
- **Execute:** This permission on a file allows it to get executed. For example, if we have a file named *php.sh* so unless we don't give it execute permission it won't run.

Types of file Permissions:

- **User:** This type of file permission affects the owner of the file.
- **Group:** This type of file permission affects the group which owns the file. Instead of the group permissions, the user permissions will apply if the owner user is in this group.
- **Other:** These types of file permission affect all other users on the system.

Note: To view the permissions we use:

```
ls -l
```

chown command is used to change the file Owner or group. Whenever you want to change ownership, you can use chown command.

Syntax:

```
chown [OPTION]… [OWNER][:GROUP] FILE…
chown [OPTION]… –reference=RFILE FILE…
```

Some examples of `chown`commands in Linux

1) To change the Owner of the file:

Syntax:

```
chown owner_name file_name
```

Now if I use *file1.txt* in my case, to change ownership I will use the following syntax:

```
chown master file1.txt
```

opendir - open a directory

```
#include <dirent.h>

DIR *opendir(const char *dirname);
```

The opendir() function shall open a directory stream corresponding to the directory named by the dirname argument. The directory stream is positioned at the first entry. If the type **DIR** is implemented using a file descriptor, applications shall only be able to open up to a total of {OPEN_MAX} files and directories.

RETURN VALUE

Upon successful completion, opendir() shall return a pointer to an object of type DIR. Otherwise, a null pointer shall be returned and errno set to indicate the error.

ERRORS

The opendir() function shall fail if:

[EACCES]

Search permission is denied for the component of the path prefix of dirname or read permission is denied for dirname.

[ELOOP]

A loop exists in symbolic links encountered during resolution of the dirname argument.

[ENAMETOOLONG]

The length of the dirname argument exceeds {PATH_MAX} or a pathname component is longer than {NAME_MAX}.

[ENOENT]

A component of dirname does not name an existing directory or dirname is an empty string.

[ENOTDIR]

A component of dirname is not a directory.

The opendir() function may fail if:

[ELOOP]

More than {SYMLOOP_MAX} symbolic links were encountered during resolution of the dirname argument.

[EMFILE]

{OPEN_MAX} file descriptors are currently open in the calling process.

[ENAMETOOLONG]

As a result of encountering a symbolic link in resolution of the dirname argument, the length of the substituted pathname string exceeded {PATH_MAX}.

[ENFILE]

Too many files are currently open in the system.

The following sections are informative.

Open a Directory Stream

The following program fragment demonstrates how the opendir() function is used.

```
#include <sys/types.h>
#include <dirent.h>
#include <libgen.h>
...
   DIR *dir;
   struct dirent *dp;
...
   if ((dir = opendir (".")) == NULL) {
      perror ("Cannot open .");
      exit (1);
   }



   while ((dp = readdir (dir)) != NULL) {

...
```

The opendir() function should be used in conjunction with readdir(), closedir(), and rewinddir() to examine the contents of the directory (see the EXAMPLES section in readdir()). This method is recommended for portability.

readdir - read a directory

```
 struct dirent *readdir(DIR *dirp);
```

The readdir() function returns a pointer to a dirent structure

representing the next directory entry in the directory stream pointed to by dirp. It returns NULL on reaching the end of the directory stream or if an error occurred.
In the glibc implementation, the dirent structure is defined as follows:

```
struct dirent {
    ino_t          d_ino;       /* Inode number */
    off_t          d_off;       /* Not an offset; see below */
    unsigned short d_reclen;    /* Length of this record */
    unsigned char  d_type;      /* Type of file; not supported
                                   by all filesystem types */
    char           d_name[256]; /* Null-terminated filename */
};
```

The only fields in the dirent structure that are mandated by POSIX.1 are d_name and d_ino. The other fields are unstandardized, and not present on all systems; see NOTES below for some further details.

The fields of the dirent structure are as follows:

d_ino This is the inode number of the file.

d_off The value returned in d_off is the same as would be returned by calling telldir(3) at the current position in the directory stream. Be aware that despite its type and name, the d_off field is seldom any kind of directory offset on modern filesystems. Applications should treat this field as an opaque value, making no assumptions about its contents; see also telldir(3).

closedir — close a directory stream

#include <dirent.h>
int closedir(DIR *dirp);
The closedir() function shall close the directory stream referred to by the argument dirp. Upon return, the value of dirp may no longer point to an accessible object of the type DIR. If a file descriptor is used to implement type DIR, that file descriptor shall be closed.

Upon successful completion, closedir() shall return 0; otherwise, -1 shall be returned and errno set to indicate the error.
The closedir() function may fail if:

EBADF The dirp argument does not refer to an open directory stream.

EINTR The closedir() function was interrupted by a signal.

The following sections are informative.

Closing a Directory Stream

The following program fragment demonstrates how the closedir() function is used.

```
...
   DIR *dir;
   struct dirent *dp;
...
   if ((dir = opendir (".")) == NULL) {
...
   }

   while ((dp = readdir (dir)) != NULL) {
...
   }
  closedir(dir);
```

mkdir command in Linux allows the user to create directories (also referred to as folders in some operating systems). This command can create multiple directories at once as well as set the permissions for the directories. It is important to note that the user executing this command must have enough permission to create a directory in the parent directory, or he/she may receive a 'permission denied' error.

Syntax:

mkdir [options...] [directories ...]

Options available in mkdir

1) –help:

It displays help-related information and exits.

Syntax:

mkdir –help

Linux File Links

A Linux file system has many hard links and symbolic links. A link is a connectivity between the filename and the actual data byte in the disk space. More than one filename can **link** to the same data.

There are two types of links in Linux OS:

1. Hard Links
2. Soft Links

---

1) Hard Links

They are the low-level links. It links more than one filename with the same Inode and it represents the physical location of a file.

When hard link is created for a file, it directly points to the Inode of the original file in the disk space, which means no new Inode is created. Directories are not created using hard links and they can not cross filesystem boundaries. When the source file is removed or moved, then hard links are not affected.

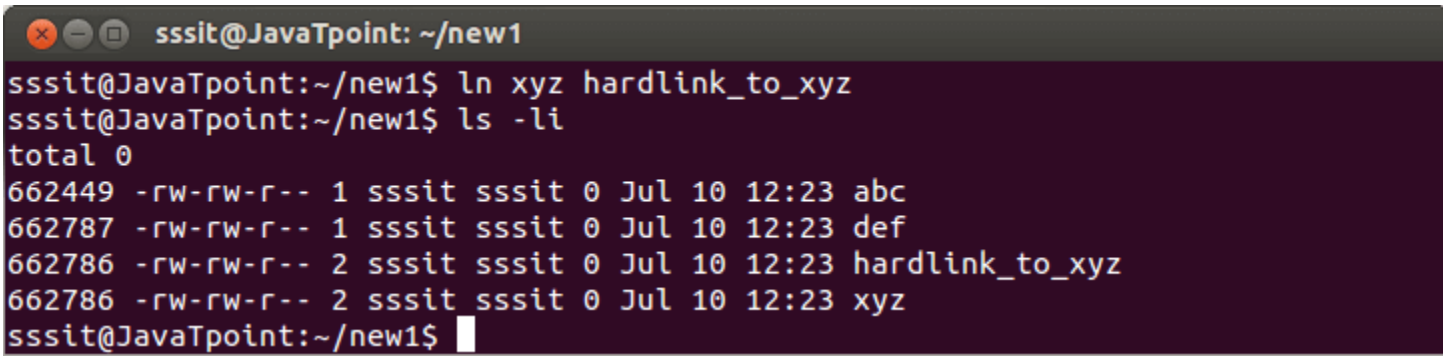
2) Soft Links (Symbolic Links)

Soft links are very common. It represents a virtual or abstract location of the file. It is just like the shortcuts created in Windows. A soft link doesn't contain any information or content of the linked file, instead it has a pointer to the location of the linked file. In other words, a new file is created with new Inode, having a pointer to the Inode location of the original file.

It is used to create link between directories and can cross filesystem boundaries. When the source file is removed or moved, then soft links are not updated.

Creating Hard Links

Hard links for any file can be created with command **ln**. One extra hard link file will be created in the respective directory.



Look at the above snapshot, we have created a hard link for the file **xyz** in the directory **new1.**

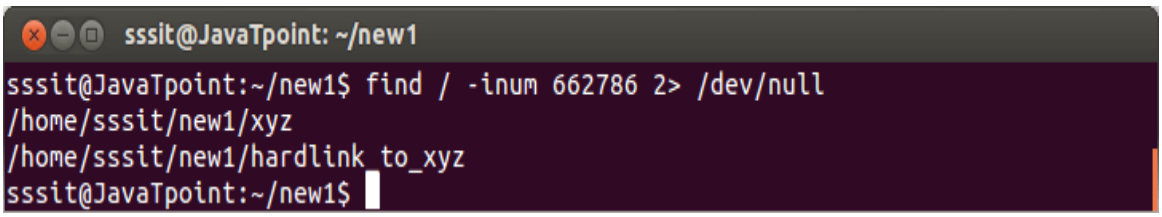
The original file and hard linked file both contain the same Inode number and hence, they have the same permissions and same owners. Content will also be the same for both the files. In short, both the files are equal now, but if original file will be removed then hard link file will not be affected.

Finding Hard Links

A hard link can be find with **find** command by specifying the Inode number. Inode number is always unique to its partition.

Example:

1. find / -inum 662786 2> /dev/null



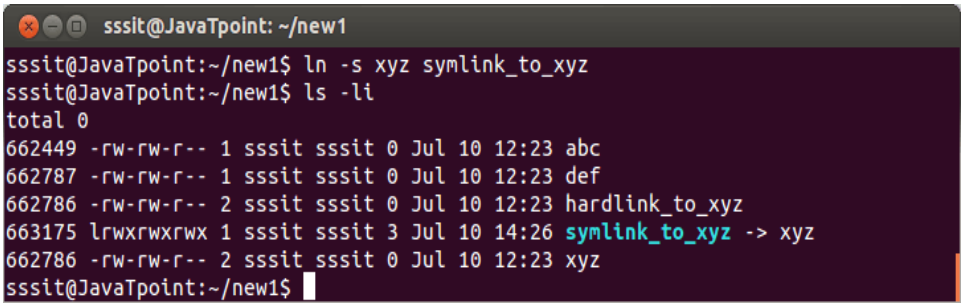
Look at the above snapshot, we have found hard link files with command **"find / -inum 662786 2> /dev/null"** for the Inode number **662786.**

Symbolic Links

Symbolic links are also called **soft links.** Command **ln -s** is used to create soft link. It doesn't link to Inodes but create a name to mapping. It create its own Inode number.

Example:

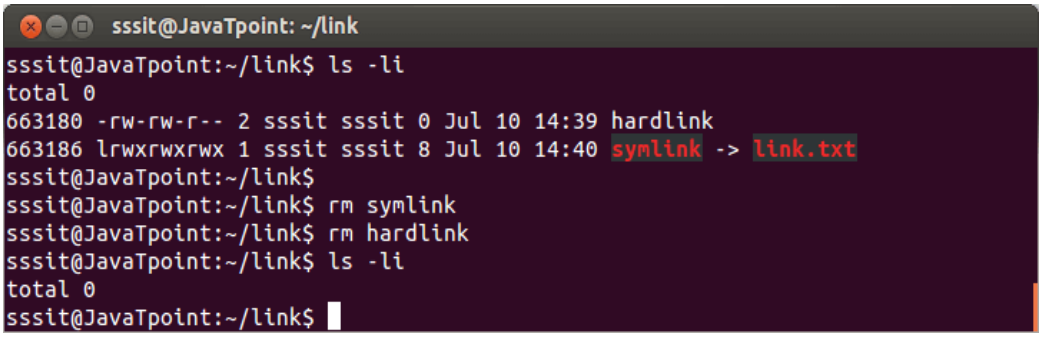
1. ln -s xyz symlink_to_xyz



Look at the above snapshot, we have created a symbolic link for file **xyz** with command **"ln -s xyz symlink_to_xyz"**. Symbolic link Inode is different from the original file Inode number. Target permissions are applied on the symlink file. Hard links are limited to their own partition, but symbolic links can be linked anywhere.

Removing Links

With **rm** command links can be removed.



Look at the above snapshot, directory **link** contains both hard link and soft link. With the command **rm** we have removed both the links.

Environment & Path Setting

How to Set the $PATH Variable in Linux

Depending on what you want to achieve---whether it's a temporary setup to run a script only once or you plan on using the script on a regular basis---there are different ways to add a directory to your $PATH.

1. Setting $PATH Variable Temporarily

If you want to run a script or a program in just your current (active) session, you can choose to set a temporary $PATH variable. When you do so, you'll be able to execute that program with a

command from anywhere on your system without having to include its full path in the command but only in the current session.

To set a $PATH variable temporarily, open the terminal and use the following syntax to add directories to the variable:

```
export PATH=$PATH:/path/to/directory
```

For example:

```
export PATH=$PATH:/home/smithy/scripts/
```

Do note that with this configuration, you can only run the program in your active session. When you set your $PATH variable temporarily, the system will reset it back to the default setting after you restart your computer.

2. Setting a Permanent $PATH Variable

If there's a program on your system that you're likely to use frequently, you must set the $PATH variable for it permanently. This will ensure that your shell remembers its directory even after a reboot.

Unlike the temporary solution, though, setting $PATH permanently involves a few extra steps: identify the shell you're using, edit the corresponding configuration file, and set the $PATH variable there.

To identify the shell on your system, run the following command in the terminal:

```
echo $0
```

You should see the name of your active shell as the output. In most cases, this will be the Bash shell, unless you've changed it to Zsh, Ksh, or some other Linux shell.

Now, based on the shell you're using, you will have to edit its configuration file. Here's what file you need to edit based on your shell:

Shell Name	Configuration File
bash	~/.bashrc
ksh	~/.kshrc
zsh	~/.zshrc
csh	~/.cshrc

In the case of some other shell, check out its documentation to find out its configuration file name.

Once you've determined the file you need to edit, follow the steps below to set the $PATH variable. In this guide, we will be demonstrating how to set up the $PATH variable in the Bash shell.

1. Open the terminal and enter the following command to open the **.bashrc** file. You can use any Linux text editor of your choice.
   ```
   nano ~/.bashrc
   ```
2. Use the following syntax to set the $PATH variable.
   ```
   export PATH=$PATH:/path/to/directory/
   ```
3. Save the file and update the current shell environment using **source**.
   ```
   source ~/.bashrc
   ```
4. Type the following command to check whether the directory has been added or not.
   ```
   echo $PATH
   ```

If you entered the directory path correctly, it will reflect in your system's $PATH, and you should be able to run all the scripts inside the folder from any directory on your system.

Besides, Linux also allows you to set the $PATH variable permanently in a system-wide configuration so that every user on your system can access and execute the custom scripts present in the specified directory.

To set $PATH globally, you'll need to add the directory using the same command you used while adding temporary and permanent setup, but either in the **/etc/environment** or the **/etc/profile** file.

Open either of these files and append the path of the directory:

```
 nano /etc/environment
nano /etc/profile
```

Successfully Setting the $PATH in Linux

With the above steps, you can set your system's $PATH to look for scripts in directories that are, otherwise, unspecified.

For most cases, we recommend you to set the $PATH variable on your system permanently in a per-user configuration since it's the most efficient way, out of the two methods.

Also, do keep in mind that unless there's a specific program on your system that you want to access globally, you wouldn't need to set the $PATH variable in a system-wide configuration. The **locate** and **find** commands in Linux are a lifesaver if you are facing issues while finding the source directory to add to the $PATH variable.

Both files `/etc/shadow` and `/etc/passwd` seem to be the same, they just report some information about accounts. What is the difference between them?

- etc
- passwd

`passwd` stores general user info and `shadow` stores user passwd info.

`passwd` is the file where the user information (like username, user ID, group ID, location of home directory, login shell, ...) is stored when a new user is created.

`shadow` is the file where important information (like an encrypted form of the password of a user, the day the password expires, whether or not the passwd has to be changed, the minimum and maximum time between password changes, ...) is stored when a new user is created.

Traditional Unix systems keep user account information, including one-way hashed passwords, in a text file called `/etc/passwd`. As this file is used by many tools (such as `ls`) to display file ownerships, etc. by matching user id #'s with the user's names, the file needs to be world-readable. Consequentally, this can be somewhat of a security risk.

Another method of storing account information, one that I always use, is with the shadow password format. As with the traditional method, this method stores account information in the `/etc/passwd` file in a compatible format. However, the password is stored as a single "x" character (ie. not actually stored in this file). A second file, called `/etc/shadow`, contains encrypted password as well as other information such as account or password expiration values, etc. The `/etc/shadow` file is readable only by the root account and is therefore less of a security risk.

/etc/shadow

file stores actual password in encrypted format for user's account with additional properties related to user password i.e. it stores secure user account information. All fields are separated by a colon (:) symbol FOR MORE

/etc/passwd

file stores essential information, which is required during login i.e. user account information. /etc/passwd is a text file, that contains a list of the system's accounts, giving for each account some useful information like user ID, group ID, home directory, shell, etc.

What is Linux passwd/groups?

It’s important to introduce two files that we will be using as examples of adding users to a group in Linux.

In the /etc directory, the passwd and the group files hold all of the users and group information. These files are essential when logging on to the system, and you'll need to know them if you ever take a Linux exam. Anytime you add a user to a group in Linux, that user is added to the passwd file.

The /etc/group file holds all of the group information, as well as the users belonging to each

group. The structure is very similar to that of /etc/password.

Components in the /etc directory and /etc/group file

1. **Issue command: less /etc/passwd**

2. **Use the arrow keys to go up and down, and q to exit.**

Viewing the directory and file

3. **Edit the file directly or use the commands below.** Use the commands, especially if you’re a beginner, to avoid accidentally corrupt the passwd file.

Linux Password & Shadow File Formats

Traditional Unix systems keep user account information, including one-way encrypted passwords, in a text file called ``/etc/passwd''. As this file is used by many tools (such as ``ls'') to display file ownerships, etc. by matching user id #'s with the user's names, the file needs to be world-readable. Consequentally, this can be somewhat of a security risk.

Another method of storing account information, one that I always use, is with the shadow password format. As with the traditional method, this method stores account information in the /etc/passwd file in a compatible format. However, the password is stored as a single "x" character (ie. not actually stored in this file). A second file, called ``/etc/shadow'', contains encrypted password as well as other information such as account or password expiration values, etc. The /etc/shadow file is readable only by the root account and is therefore less of a security risk.

While some other Linux distributions forces you to install the Shadow Password Suite in order to use the shadow format, Red Hat makes it simple. To switch between the two formats, type (as root):

```
/usr/sbin/pwconv     To convert to the shadow format
/usr/sbin/pwunconv   To convert back to the traditional format
```

With shadow passwords, the ``/etc/passwd'' file contains account information, and looks like this:

```
smithj:x:561:561:Joe Smith:/home/smithj:/bin/bash
```

Each field in a passwd entry is separated with ":" colon characters, and are as follows:

- Username, up to 8 characters. Case-sensitive, usually all lowercase
- An "x" in the password field. Passwords are stored in the ``/etc/shadow'' file.
- Numeric user id. This is assigned by the ``adduser'' script. Unix uses this field, plus the following group field, to identify which files belong to the user.
- Numeric group id. Red Hat uses group id's in a fairly unique manner for enhanced file security. Usually the group id will match the user id.
- Full name of user. I'm not sure what the maximum length for this field is, but try to keep it reasonable (under 30 characters).
- User's home directory. Usually /home/username (eg. /home/smithj). All user's personal files, web pages, mail forwarding, etc. will be stored here.
- User's "shell account". Often set to ``/bin/bash'' to provide access to the bash shell (my personal favorite shell).

Perhaps you do not wish to provide shell accounts for your users. You could create a script file called ``/bin/sorrysh'', for example, that would display some kind of error message and log the user off, and then set this script as their default shell.

Note: Note: If the account needs to provide "FTP" transfers to update web pages, etc. then the shell account will need to be set to ``/bin/bash'' -- and then special permissions will need to be set up in the user's home directory to prevent shell logins.

The ``/etc/shadow'' file contains password and account expiration information for users, and looks like this:

```
smithj:Ep6mckrOLChF.:10063:0:99999:7:::
```

As with the passwd file, each field in the shadow file is also separated with ":" colon characters, and are as follows:

- Username, up to 8 characters. Case-sensitive, usually all lowercase. A direct match to the username in the /etc/passwd file.
- Password, 13 character encrypted. A blank entry (eg. ::) indicates a password is not required to log in (usually a bad idea), and a ``*'' entry (eg. :*:) indicates the account has been disabled.
- The number of days (since January 1, 1970) since the password was last changed.
- The number of days before password may be changed (0 indicates it may be changed at any time)
- The number of days after which password *must* be changed (99999 indicates user can keep his or her password unchanged for many, many years)

- The number of days to warn user of an expiring password (7 for a full week)
- The number of days after password expires that account is disabled
- The number of days since January 1, 1970 that an account has been disabled
- A reserved field for possible future use

Ways to Add a User to a Group in Linux

1. Log in as root
2. Use the command useradd "name of the user" (for example, useradd roman)
3. Use su plus the name of the user you just added to log on.
4. "Exit" to log out

Alternative:

1. Use the usermod command.
2. Replace examplegroup with the name of the group.
3. Replace exampleusername with the name of the user you want to add.

How to delete a user from Linux

There are several options to be mindful of as you delete a user from a group in Linux:

- Use the command userdel “name of the user” (for example, userdel roman).
- The command -d sets home directory for the user (if it’s different than the default which is /home/”user’s name”). Using -d on its own will only set the home directory for the user, but does not create it. You can see this is confirmed by “echo $HOME” which tells me my home directory. I use “ls” to confirm.

Using userdel and checking to see if you created a home directory

- Use the command -m to create the home directory.
- If you just add the user, the default directory is /home/“user’s name”. You can just use the -m to create.
- Use the -r command along with userdel to delete the user as well as the home directory.

How to Add a Group in Linux

To create a new group in Linux, follow these steps:

1. Use the groupadd command.
2. Replace new_group with the name of the group you want to create.
3. Confirm by checking the /group/etc file (for example, grep software /etc/group or cat /etc/group).
4. Run the grep command to confirm.

Using groupadd and confirming creation

If you want to add the user roman to the group “accounting” and make his primary group “sales,” do this:

1. Use the command -G for other groups (-G adds the user to a new group but also keeps them in the old one (append).

   Note: Make sure you use a capital (-G) with this command to append a new group. If you do not, it will assign that group as primary and not add you to other groups.
2. Use the command id roman to confirm

How to Change a User’s Group in Linux

If a user is already created and you just want to add that user to a group:

- Use the command usermod -Gmanagement “name of user”

If you want to change the primary group a user is assigned to:

- Use the command usermod.
- Replace the examplegroup with the name of the group you want to be the primary.
- Replace the exampleusername with the name of the user account.

If you want to change a user’s home directory:

- Use the command usermod -d/home/newfolder “name of user”.

The command usermod -d/home/newfolder "name of user"

How to Change Passwords in Linux

To change passwords in Linux:

1. Login as root.
2. Use the command passwd "username" (for example, passwd roman).

If you are logged on as the user, the command is passwd.

The next command, `useradd` will work across RHEL based OS distributions as well as works equally well on Ubuntu/Debian hosts. The simplest syntax (without any extra options) to create a new user is:

```
$ sudo useradd <username>
```

Example:

```
$ sudo useradd janedoe
```

The `useradd` command supports multiple options that can be specified while creating the user, most common being user ID (UID), group ID (GID), default shell and home directory, etc. One such example is given below:

```
$ sudo useradd -s /bin/sh -d /data/newhome -c "Jane Doe" -u 1005 janedoe
```

You can verify the newly created user using `id` command:

```
$ id janedoe
uid=1005(janedoe) gid=1005(janedoe) groups=1005(janedoe)
$
```

Modifying User

One often needs to modify some property of existing users based on organization requirements, user requests, or system migrations. Most of these properties are easy to modify though we need to ensure how it'll affect the user environment and access to files owned or accessed by the user.

Default Shell

The default shell is the CLI shell created when a user launches a new CLI session either locally or via SSH. Most modern systems have a default user `Bash` though it can vary based on Linux distribution or the user's environment. To modify the default shell of a user, use:

```
$ sudo usermod -s <shell> <username>
```

Example:

```
$ getent passwd janedoe
janedoe:x:1005:1005::/data/newhome:/bin/sh
$ sudo usermod -s /bin/bash janedoe
$ getent passwd janedoe
janedoe:x:1005:1005::/data/newhome:/bin/bash
$
```

As you can see in the above output, the shell has been changed from `/bin/sh` to `/bin/bash` for user `janedoe`.

Home Directory

Like default shell, a user's home directory can be modified to a different location using:

```
$ sudo usermod -d <new_directory_path> <username>
```

In the example below, the user home directory of the user `janedoe` has been changed to `/data/janedoe`:

```
$ getent passwd janedoe
janedoe:x:1005:1005::/data/newhome:/bin/bash
$ sudo usermod -d /data/janedoe janedoe
$ getent passwd janedoe
janedoe:x:1005:1005::/data/janedoe:/bin/bash
$
```

Before making the switch, ensure that the new directory has the right ownership and permissions. Otherwise, the user may face issues during login or working in the new home directory.

User ID

You can change the user ID of an existing user using:

```
$ sudo usermod -u <new_uid> <username>
```

Example:

```
$ getent passwd janedoe
janedoe:x:1005:1005::/data/janedoe:/bin/bash
$ sudo usermod -u 1010 janedoe
$ getent passwd janedoe
janedoe:x:1010:1005::/data/janedoe:/bin/bash
$
```

Again, changing UID changes how Linux filesystem maps ownership and permission to a file or directory. Ensure that the user's home directory and its contents and all other files anywhere in the system, originally owned by the user (with old UID), is changed to UID mapped. Not doing so can cause problems in the CLI session and file access by the user.

Default Group

The default group is usually the user's default group ID, which gets created during user creation unless another GID is specified. Linux allows you to modify the default group of a user using `usermod` command as well. Here's the syntax to use:

```
$ sudo usermod -g <new_gid or group_name> <username>
```

Here's one example:

```
$ getent passwd janedoe
janedoe:x:1010:1005::/data/janedoe:/bin/bash
$ sudo usermod -g 1001 janedoe
$ getent passwd janedoe
janedoe:x:1010:1001::/data/janedoe:/bin/bash
$
```

Again, make sure the new group ID is set on the user's home directory, contents, and all other files or directories applicable to properly migrate their ownership permissions.

Adding/Removing Groups

Besides the default group, a user in Linux can be part of secondary groups. We can always add or remove additional groups a user belongs to using `usermod` command.

```
$ sudo usermod -a -G <group_id or group_name> <username>
```

Example:

```
$ id janedoe
uid=1005(janedoe) gid=1005(janedoe) groups=1005(janedoe)
$ sudo usermod -a -G docker janedoe
$ id janedoe
uid=1005(janedoe) gid=1005(janedoe) groups=1005(janedoe),1001(docker)
$
```

Similarly, to remove a user from one of the secondary groups, use `gpasswd` command as shown below: